UNIT - III

UNIT - III

Agile Modelling and XP: Introduction

The Fit

Common Practices

Modelling Specific Practices

XP Objections to Agile Modelling

Agile Modelling and Planning XP Projects

XP Implementation Phase.

Agile Modelling and XP

Over the last four chapters, we have considered in detail both Agile Modelling (AM) and eXtreme Programming (XP). Both are from the agile movement and both are motivated by the desire to produce better software faster. But how do they relate (if at all)? What is the relationship between Agile Modelling and XP? Are they complementary or contradictory?

In this UNIT, we will consider exactly this issue and look at how Agile Modelling can actually enhance an XP project

– The Fit

- In actual fact, many of the Agile Modelling practises fit straight into XP. In many cases, they are the modelling equivalent of the XP programming-oriented practices.
- It is also important to consider whether Agile Modelling actually fits in with the philosophy of XP at all. At first sight, there may be a fundamental conflict
- Agile Modelling needs to be applied with some other development methodology to be of any real benefit. As such, it brings a suite of agile techniques to the process of modelling. XP does do modelling, although this may come as a shock to some alleged XP practitioners. However, it does not do “big up-front modelling.” Instead, modelling is performed as and when required during the lifetime of an XP project

Common Practices

- So both Agile Modelling and XP are part of the agile movement so what practices do they have in common. Actually, they have quite a few practices in common (although the names may differ from one approach to the other). To illustrate this, consider Table 7.1. This table pairs up various Agile Modelling practices with their equivalent (or near-equivalent) XP practices.

Table 7.1 Common practises between Agile Modelling and eXtreme Programming

Agile Modelling	*eXtreme Programming*
Collective ownership	Collective ownership
Create simple content/depict models simply	Simple design
Model with others	Pair programming
Apply modelling standards	Use coding standards
Active stakeholder participation	On-site customer

- Other Agile Modelling practises have potentially less obvious parallels within XP practices. But again if they are examined, it can be seen that they represent the same or similar intention but from the modelling perspective. These are illustrated in Table 7.2.
- For example, in Table 7.2 when performing Agile Modelling you should consider how what you are modelling might be tested? How you can model to facilitate testing? (and by implication what those tests are

Table 7.2 Parallel practises between Agile Modelling and eXtreme Programming.

Agile Modelling	*eXtreme Programming*
Consider testability	Testing (test-first coding)
Display models publicly	Open communication
Model in small increments	Implement a test at a time
Prove it with code	Pass a test before continuing

Modelling Specific Practices

There are some Agile Modelling practices that may, at first at least, appear to have no place within XP at all. These practices include:

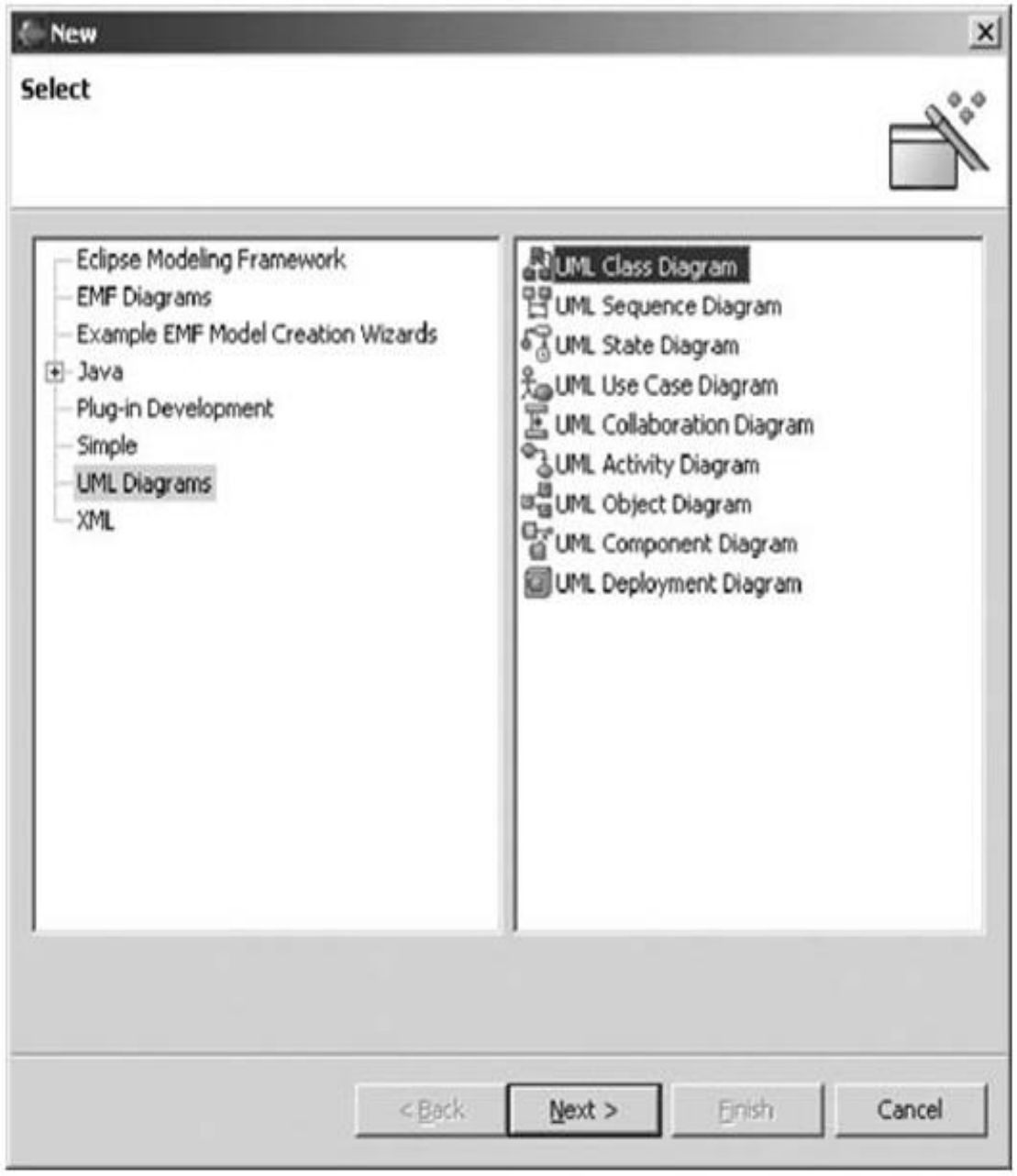
- model with a purpose,
- using multiple models and
- know your models.

This is partly as they are very clearly specific to the act of modelling. It is also partly because XP does not have much to say about the act of modelling or of the models that might be created. The only thing that tends to be discussed is the use of stand-up modelling meetings during which existing code is analyzed or new code explored through the use of diagrams (which are a form of model).

Model with a Purpose

As many XP practitioners may see no place for modelling at all, this point may, to them, seem self-defeating. That is, there is no purpose to modelling within an XP project! However, this can be very wrong. There are a number of purposes to modelling which can be very relevant to XP. For example, the motivational category within Agile Modelling that includes the practices "Model to Understand" and "Model to Communicate." These two practices are considered in more detail below.

- *Model to understand the software*. This is relevant because, before you refractor or extend your code you must understand it. Individual methods or classes may be understandable merely by considering them in isolation



For example, when looking at part of the implementation of the Clear Spell spell checker developed by a colleague, I need to become familiar with the structure of the main spell-checking engine. To do this, I have decided to use the Omondo plug-in for Eclipse to generate a class diagram for myself. This is done within Eclipse by creating a new UML diagram as illustrated in Figure 7.1.

Fig. 7.2 Selecting the contents of the class diagram.

Once this is done, Omondo finds out which classes and interfaces are in the selected package. It then presents a selection dialog to the user, which allows them to select which classes and interfaces they actually want to include in the diagram and what type of relationships to show. This is illustrated in Figure 7.2.

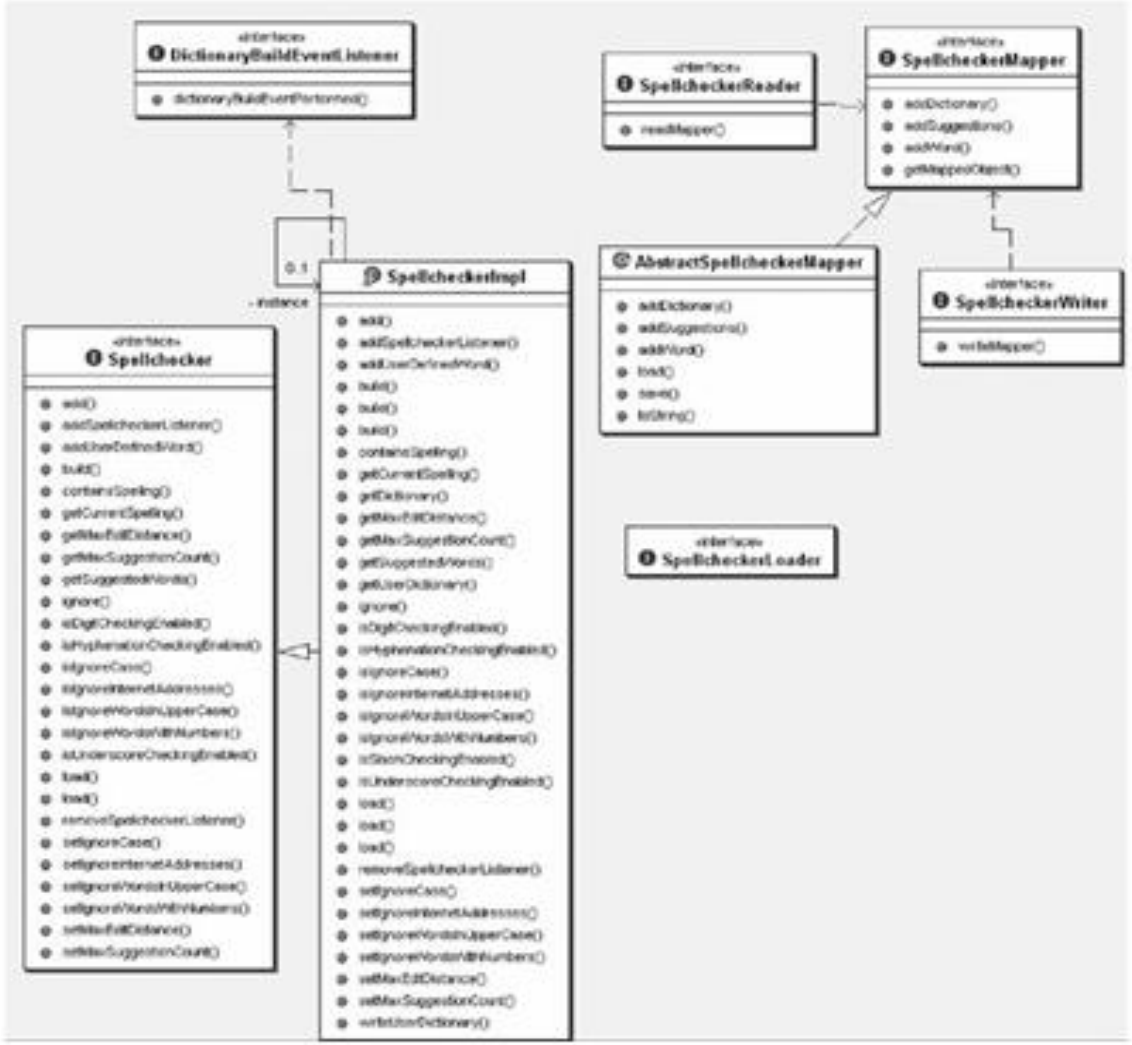


Fig. 7.3 The auto-generated class diagram.

Having done this, Omondo then creates the class diagram displayed in Figure 7.3.

Fig. 7.4 Listing the package contents.

Although I must still determine what the various elements of this package now do, I have a much better feel for the structure of this package, then I would have got just by looking at what the package contains (as illustrated in Figure 7.4).

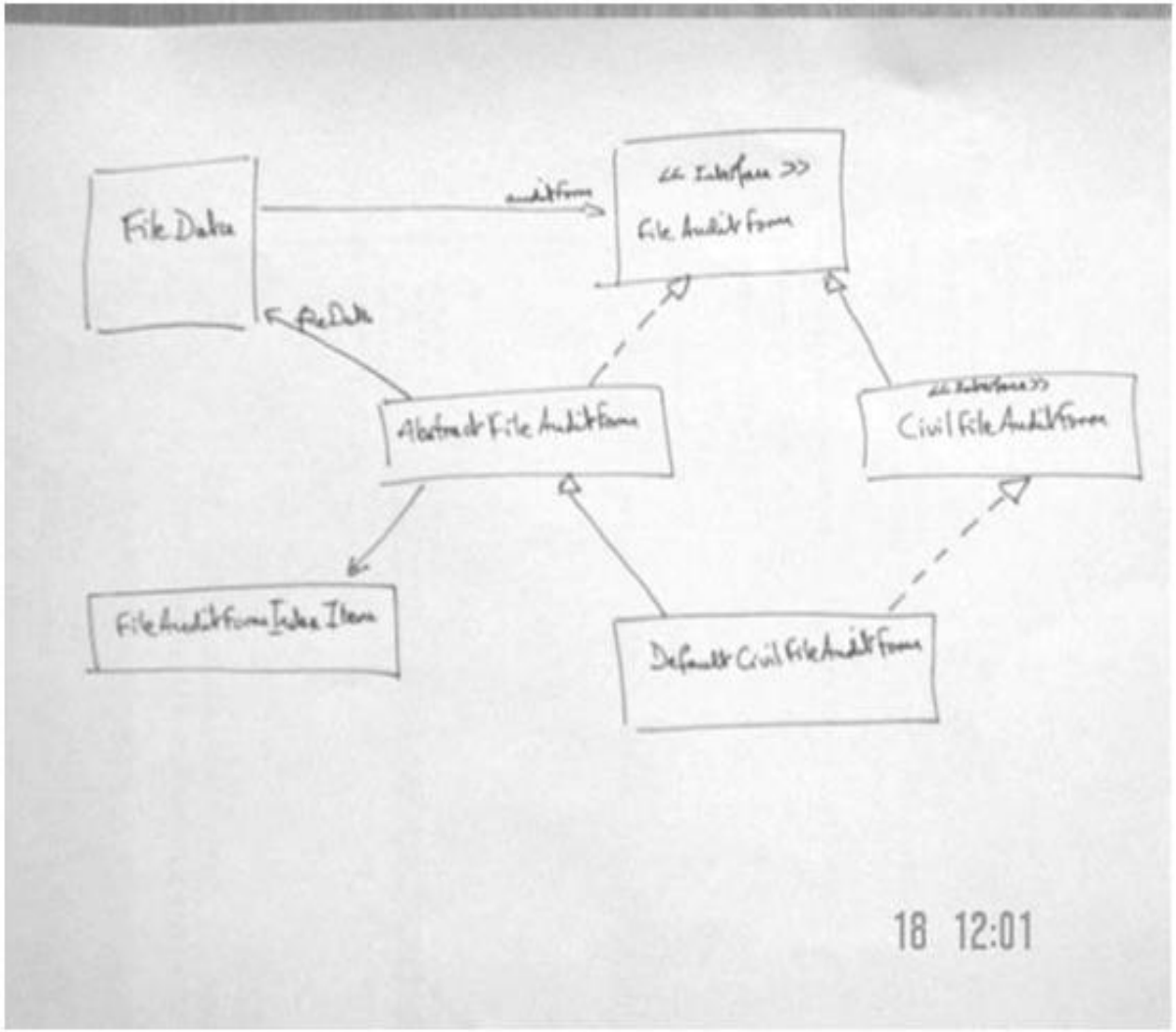


Fig. 7.5 Hand drawn class diagram.

Of course, I do not need to use a tool such as Eclipse or Together at all. I might find that merely drawing out the structure on a white board is more than sufficient (as illustrated in Figure 7.5). *Model to communicate.* Source code is the end result that we all are trying to produce (it is not a model which will be delivered as an executable after all). However, it is not the best way to explain your ideas to one or more other people

Know Your Models

Whether you are an Agile modeller or an XP developer you need to know the tools available to you. Even for those working on an XP project, modelling is relevant (as has already been said). Thus, XP developers need to understand the models available to them just as much as an Agile modeller should. That is, they should understand the strengths and weaknesses of different types of models. This will help them to keep models as simple as possible, as well as helps to apply appropriate models which will help them to understand the systems under consideration

XP Objections to Agile Modelling

Other arguments raised by XP practitioners against Agile Modelling include:

- *Modelling is all about big up-front design* (the so called BMUF – Big Modelling Up-Front syndrome). Agile Modelling clearly does not promote this. This is illustrated by the Agile Modelling practices such as “Model in Small Increments” and “Prove it with Code.”

- *All models are permanent documents* that must be updated when any changes are made. This is clearly not what Agile Modelling says. For example, the practises "Discard temporary models," "Use the simplest tools" and "Update only when it hurts" contradict this view.

- *You need to use a complex modelling tool*, such as Rational Rose to carry out any modelling activity. However, as Agile Modelling explicitly debunks that myth, stating you should use whatever modelling medium is appropriate, which may include modelling tools such as Rational Rose, but also white boards, index cards, post-it notes, etc.

- *You need to know, and use, UML to create models*. Agile Modelling does say that you should know how to apply whatever representation you are using in your models and that UML is one example of this. But it does not mandate any particular type of representation and indeed Agile modellers know that something like UML does not cover all the modelling situations you might want. In addition, an Agile modeller will not worry about creating a precise and complete UML diagram. Instead, they will focus on the audience of what they are creating and make sure that it is comprehensible to that audience.

- *XP does not encourage modelling*. Actually this is wrong. XP does promote the creation and use of models. The use of index cards for user stories and classes is a form of modelling. XP practitioners will also often draw diagrams on white boards while trying to consider how to address a problem of refactor code, etc. These are again forms of models.
- *XP does not create any documentation* and models are a form of unnecessary documentation. XP promotes code as the core form of documentation for a system as only code is in sync with code. However, documentation needs to be appropriate for the reader of that documentation. Source code may be a good source of reference for programmers, but it is unlikely to be appropriate for end users, non-programmers, support personnel, etc. In some cases, models may be a very useful form of documentation for some of these audiences.

Agile Modelling and Planning XP Projects

- In this section, we will consider how and where Agile Modelling fits into the project-planning aspects of an XP project. An XP project is planned at a number of levels and at various points during an XP projects lifetime (see Figure 7.6). This means that Agile Modelling practices may be more or less relevant at different stages during this process. In the following, we will consider the planning process and where Agile Modelling can be exploited to the benefit of XP

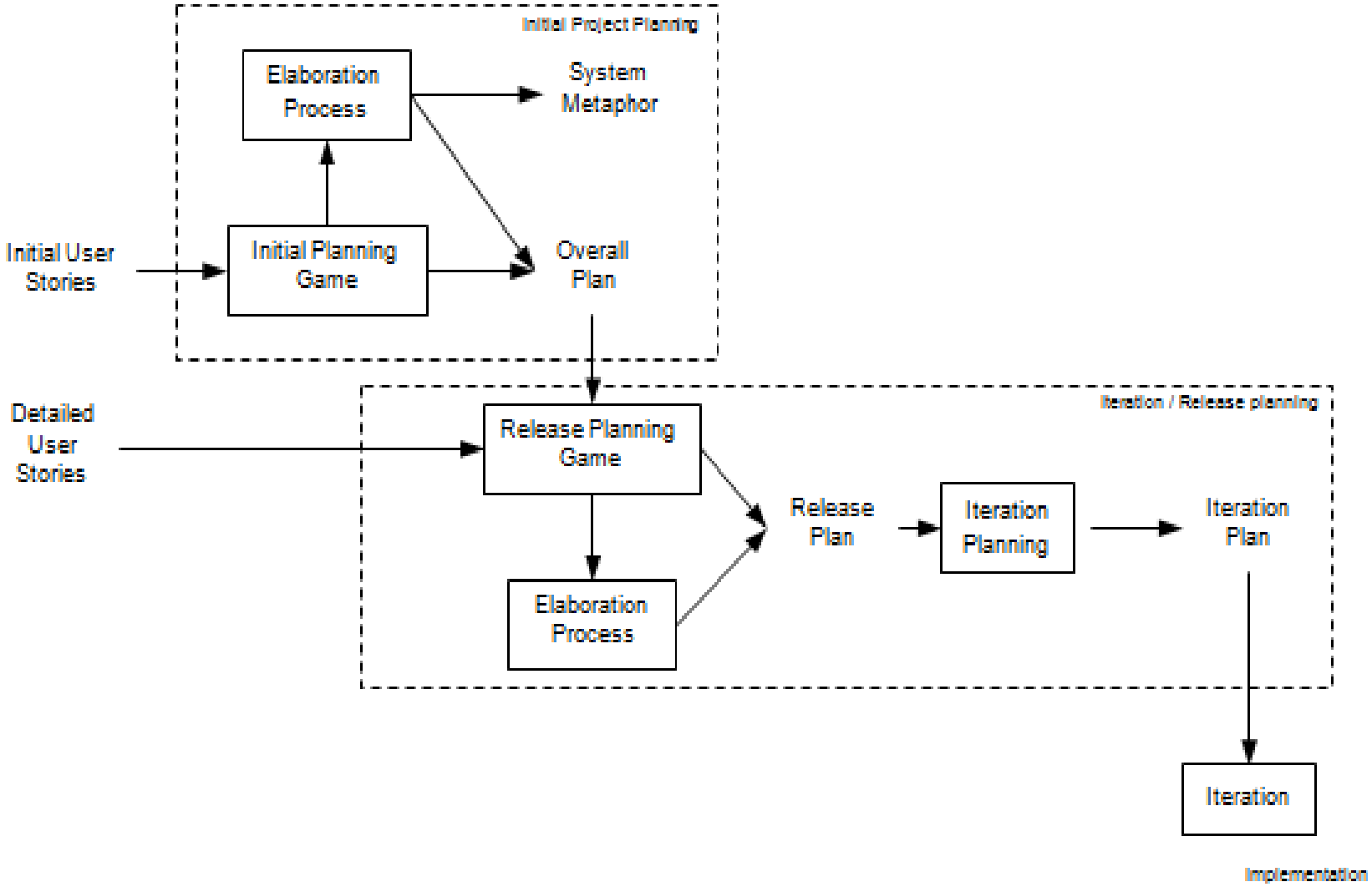


Fig. 7.6 The planning aspects of an XP project lifecycle.

Initial Project Planning

- There are two primary steps within the initial project-planning phase; these are the initial planning game and the elaboration process.
- *Initial planning game*. During this process, business and development may resort to modelling to help them clarify the user stories. By applying Agile Modelling practices, this modelling can be controlled and focused. An example of where they might do this is when a User Interface mock up might be created, with some simple flow charts to prototype system behaviour as a way of elaborating a user story.
- *Elaboration process*. Duringthe elaboration process, various models may be created to help the developers understand what will be required of the system. This will help to produce better estimates, etc. Again Agile Modelling practices can be of great help here.

Iteration/Release Planning

- During the iteration/release planning stages, modelling is again important.
- *Release planninggame*. As with the initial planning game, Agile Modelling practices can help focus the modelling activities used to clarify user requirements.
- *Elaboration process*. Although this is typically a shorter process than for the initial project planning phase, some modelling often still takes place and Agile Modelling can be applied to ensure that modelling does not become a burden.
- *Iteration planning*. In order to break down user stories into tasks, it may be necessary to model how the user stories might be implemented. This might involve initial class structures, behavior, etc. This can allow tasks to be identified, clarified or split up. Note that this is not large up-front design, as the models may be discarded and may only be intended to help elaborate the tasks.

XP Implementation Phase

- This is where the code actually gets written within an XP project. There are therefore various points at which a model may be relevant and therefore Agile Modelling practises may be applied. For example, in helping to understand code in order to refactor it, etc. We will now look at how Agile Modelling can comple- ment several of the implementation-oriented practises of XP. By implementation- oriented practises, I mean practises such as “Test-first coding,” and “Refactoring,” rather than the more process-oriented ones such as, “The Planning Game” or the “40 hour week rule.”
- The practises to be looked at in this section are:
- refactoring,
- test-first coding,
- simple design and
- pair programming.

Refactoring

- Refactoring is primarily a code improvement technique, so is it compatible with a modelling activity? Is modelling and Agile Modelling in particular, relevant to refactoring? The answer of course is “yes,” as we have already indicated earlier in this chapter. In the last chapter, some of the issues to consider when refactoring were given as:
- Make sure you know how to improve the code.
- Make sure what you have done has improved the code.
- That is, “know what you are doing!” It was also stressed that you should not refractor:
- “When you haven’t got a clear plan of how you will improve the code.”

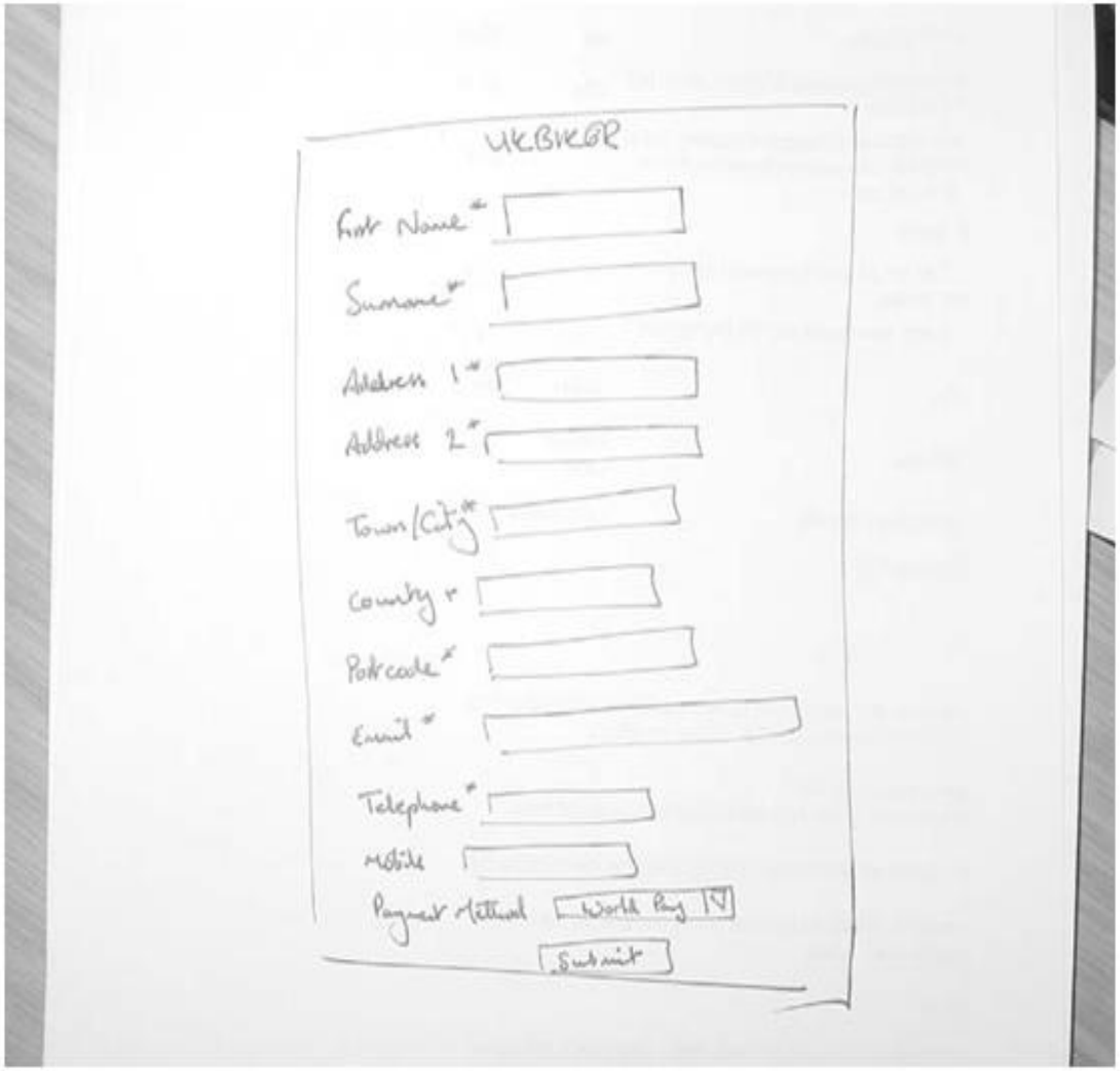


Fig. 7.7 User interface design.

For example, Figure 7.7 illustrates a possible user interface design for a membership web site. This is a diagram drawn on a white board to consider what fields are needed and what will happen when a user selects the submit option.

Test-First Coding

- At first sight, with respect to test-first coding, modelling may seem at best superfluous and at worst contradictory. This is because, in test-first coding, you essentially follow this cycle:
- Write a test.
- Write the code to be tested.
- Run the test/get the code to work.
- If the test has passed, then return to step 1 until finished.

- Thus, if you modify the test-first coding cycle to the following, then you are maximizing this XP practice, as well as, supporting the Agile Modelling principle of rapid feedback:
-
- Write a test.
- Model the solution.
- Implement the solution.
- Run the test/get the code to work.
- Discard temporary models.
- If the test has passed, then return to step 1 until finished.

Simple Design

- The XP practice of simple design aims to promote the simplest implementation that will:
- Run all the tests.
- Has no duplicate code.
- Makes it clear to anyone reading the code what it is meant to do.
- Have the fewest possible classes and methods.
- There are in fact a variety of Agile Modelling practices that also promote simplicity within modelling. These are:

- Create simple content.
- Depict models simply.
- Apply patterns gently.
- Formalize contract models.

Pair Programming

Pair programming, as we know, involves two developers working together at a single machine. Within the pair, one developer controls the mouse and keyboard (the driver) while the other essentially monitors what the first is doing (the navigator). It is essentially that the pair communicates and understands what they are trying to achieve. It is also important that the navigator understands how the driver is intended to achieve their goals.

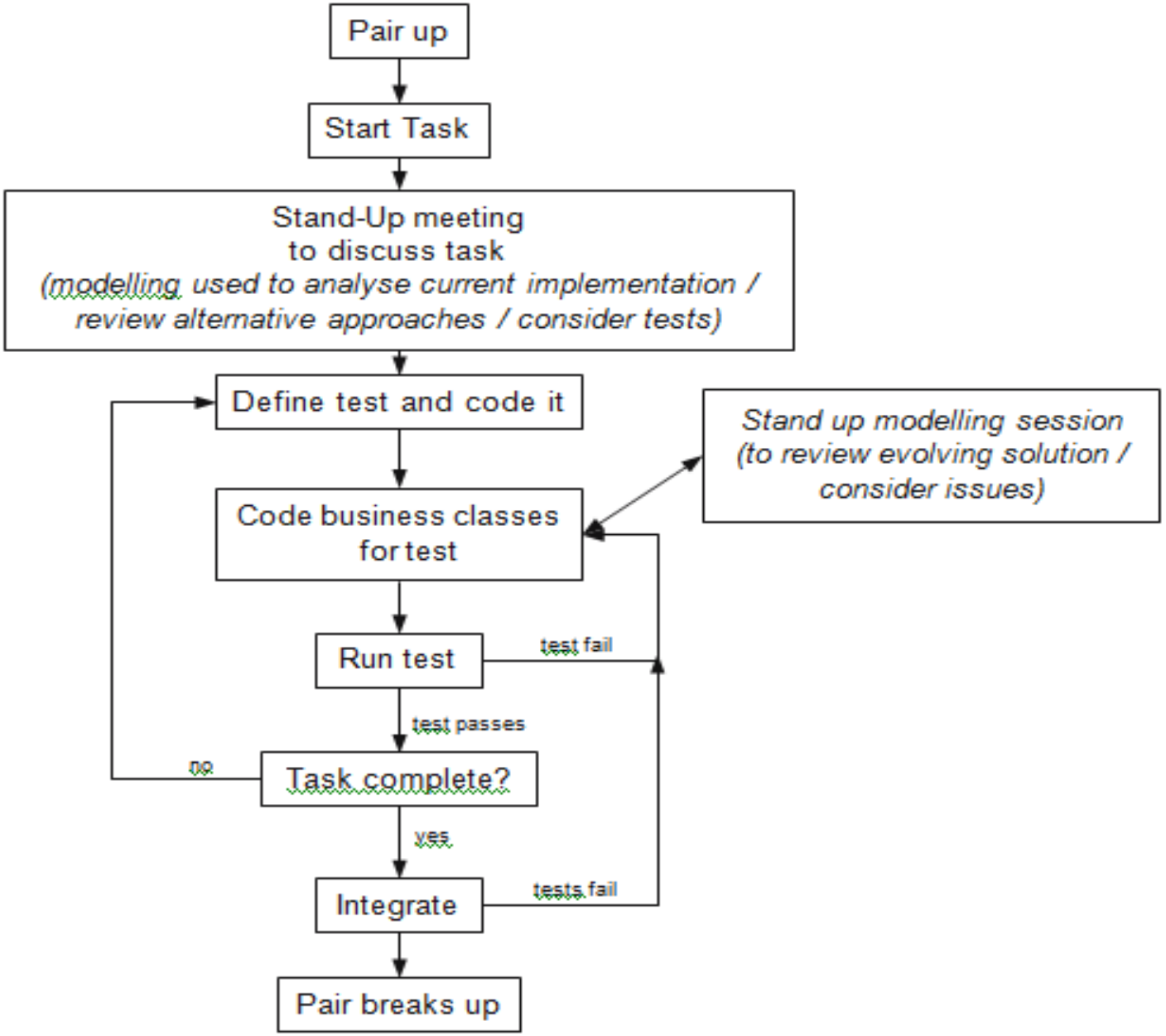


Fig. 7.10 Pair-programming workflow annotated with modelling steps.

UNIT-III

- What is Extreme Programming (XP)?
- Who developed XP and in what context?
- How does XP align with the Agile Manifesto?
- What types of software development projects are best suited for XP?
- What are the main goals of XP?
- How does Agile Modelling (AM) complement Extreme Programming (XP)?
- What modelling gaps does AM fill in XP practices?
- Why is Agile Modelling referred to as a "fit" for XP rather than a replacement?
- In what ways does Agile Modelling enhance communication in XP?
- How can combining AM and XP improve the quality of deliverables?
- What are the common development practices shared by AM and XP?
- How do AM and XP both promote simplicity?
- How is customer collaboration emphasized in both AM and XP?
- Explain how iterative development is practiced in both AM and XP.
- How do both methodologies address the issue of documentation?
- What are the core principles of Agile Modelling?
- Explain “Model with a Purpose” and why it is important.
- What is meant by “Travel Light” in Agile Modelling?
- How does Agile Modelling use “Just Barely Good Enough” models?
- What is “Multiple Models” in AM and how does it help in development?

- Why might some XP practitioners be resistant to Agile Modelling?
- What are the common objections from the XP community regarding AM?
- How can Agile Modelling address concerns of over-modelling in XP?
- Are Agile models seen as contrary to the XP principle of simplicity?
- How can the Agile Modelling approach be adapted to address XP's focus on working code?
- How does Agile Modelling assist in planning XP projects?
- In what way does AM support the Planning Game in XP?
- How can models be used to derive estimates and stories in XP?
- What is the role of modelling in release and iteration planning?
- How can modelling activities remain lightweight during XP planning?
- What is involved in the implementation phase of XP?
- How do modelling activities evolve during the XP implementation phase?
- How are Agile models kept up-to-date during continuous development?
- How is code quality maintained through practices like refactoring in implementation?
- What roles do unit testing and integration play in the XP implementation phase?